

Development of a Web-Based Self-Driving Car Rental Support Platform Utilizing Java Servlet and SQL Server

First Author¹, Second Author¹, and Third Author¹

FPT University, Da Nang, Vietnam
{auth1, auth2, auth3}@fpt.edu.vn

Abstract. The rapid expansion of the sharing economy has drastically transformed traditional personal vehicle utilization frameworks. This paper details the structural design and deployment of a comprehensive web-based self-driving car rental support platform built upon a Model-View-Controller (MVC) architecture leveraging Java Servlet, JSP, and SQL Server backend infrastructures. The application focuses on automating vehicular contract flows, optimizing real-time fleet availability, and analysing precise battery metrics during handover cycles. Secure electronic transactions are simulated seamlessly via the VNPAY Sandbox framework to handle deposit configurations and automated user refunds. The system additionally integrates AI-powered KYC OCR for identity document extraction and an AI support chatbot for first-line customer assistance. Experimental testing metrics confirm that the platform significantly optimizes transaction integrity and system efficiency across all five core operational stakeholder roles.

Keywords: Self-driving car rental · Java Servlet · VNPAY API · Fleet control · KYC verification · E-commerce architecture.

1 Introduction

1.1 Motivation and Problem Statement

Modern internet-driven ecosystems have successfully facilitated distributed peer-to-peer sharing dynamics within metropolitan transportation segments. In recent years, the personal vehicle rental market in Vietnam has grown rapidly, driven by increasing demand for flexible and affordable transportation alternatives to ride-hailing and public transport. However, connecting vehicle owners who wish to monetise idle assets with customers seeking reliable self-drive rental options remains largely fragmented and manual, relying on phone calls, social media groups, or informal brokers [4].

Conventional car rental operations are bounded by fragmented information channels that expose vehicle fleets to logistical uncertainties regarding battery capacity reporting, availability indexing, and transparent security deposit handling. Existing solutions on the Vietnamese market either focus exclusively on

ride-hailing (requiring a professional driver) or are limited to large fleet operators, leaving a gap for a peer-to-peer self-drive rental platform tailored to individual vehicle owners and renters.

1.2 Project Scope

To counter these operational problems, this research introduces an enterprise-grade web platform utilizing Java Servlet, JSP, and relational SQL Server. The in-scope deliverables are:

- **Platform:** Web application following MVC architecture using Java Servlet, JSP and SQL Server.
- **Fleet Management:** Vehicle listings, real-time availability, fuel/battery level recording at handover, and automated rental contract processing.
- **Payment Services:** Online rental booking, automated security deposit handling, and payment gateway integration via VNPAY Sandbox [1].
- **AI Augmentation:** KYC OCR for identity document extraction and an AI support chatbot for user assistance.

1.3 Paper Structure

The remainder of this paper is organised as follows. Section 2 models the platform stakeholders and presents a competitor analysis. Section 3 maps all 43 functional use cases. Section 4 presents the system high-level design including MVC architecture, database schema, and code package structure. Section 5 describes the screen flow and authorisation matrix. Section 6 concludes and outlines future work.

2 System Architecture and Stakeholder Modelling

2.1 Actors

The platform defines five major stakeholder roles:

1. **Guest:** Unauthenticated visitor. May browse the vehicle catalogue, view daily pricing, read platform policies, and read public reviews.
2. **Renter:** Authenticated account holder. May book vehicles, manage trips, pay deposits, track rental status, and leave reviews.
3. **Car Owner:** Fleet partner who has passed KYC. May register vehicles, confirm or reject booking requests, and view revenue reports.
4. **Administrator (Admin):** System-wide manager. Controls user accounts, approves vehicle listings, oversees payment flows, and manages system configuration.
5. **Customer Service (CS):** Specialised support staff. Handles KYC verification, support tickets, complaint resolution, incident management, and review moderation.

2.2 Competitor Analysis

Table 1 benchmarks the proposed platform against two established players in the Vietnamese self-drive rental market.

Table 1. Competitive Analysis Against Market Alternatives.

Criteria	Our Platform	MiCar	Xe Tu Lai
Model	P2P: owners list, renters book	Managed fleet	Traditional agency
Booking	Fully online incl. handover	e- Online + call con- firm	Phone / form
Payment	VNPAY + in-app wal- let	Online + cash de- posit	Cash/transfer
KYC	Built-in with CS re- view	Manual at pickup	Manual at pickup
Handover	Digital form + photos	Paper / chat	Paper-based
Incident	In-app with CS assign- ment	Phone/email	Phone only
Strengths	End-to-end digital; P2P supply; fraud reduction	Established brand; reliable fleet	Low cost; local service
Weaknesses	New platform; building required	trust Not P2P; price	higher No digital records; not scalable

3 Functional Use Case Mapping

The platform implements 43 use cases grouped into seven functional blocks (Table 2).

Table 3: Complete Use Case Descriptions (UC01–UC43).

ID	Feature	Use Case	Description
UC01	Auth	Register count	Ac- Guest registers by providing Email, Full Name, and Password. System sends a verification token link to activate the account.
UC02	Auth	Verify Email	System sends a verification email; user clicks the token link to activate before login is permitted.

(Continued from previous page)

ID	Feature	Use Case	Description
UC03	Auth	Login	User logs in via Email/Password or Google OAuth; system validates credentials and redirects to the role dashboard.
UC04	Auth	Login with Google	Guest authenticates via Google OAuth; system auto-creates or links the user account.
UC05	Auth	Logout	User ends the session via Navbar; session token is cleared to protect account security.
UC06	Auth	Forgot word	User requests password reset by entering registered email; system sends a time-limited reset link.
UC07	Auth	Reset word	User accesses the reset link and sets a new password; system validates the token first.
UC08	Auth	Change word	Logged-in user updates password from account settings by confirming old and new passwords.
UC09	Account	View & Edit Profile	User views and updates personal information: name, phone, avatar, date of birth.
UC10	Account	KYC Verification	User uploads National ID and Driver's Licence; AI OCR pre-extracts fields; CS approves or rejects.
UC11	Account	Link Bank Account	Owner or Renter links a bank account to the platform wallet for withdrawals and refunds.
UC12	Account	Register as Owner	Renter upgrades to Car Owner role by submitting required information; CS reviews and approves.
UC13	Vehicle Browse	View Vehicle List	Guest/Renter views available vehicles with name, price per day, location, and thumbnail.
UC14	Vehicle Browse	Search & Filter	User filters vehicles by price range, brand, category, location, seats, and fuel type; results update in real time.
UC15	Vehicle Browse	Vehicle Detail	User views full specs, images, features, rental terms, owner info, and previous reviews.

(Continued from previous page)

ID	Feature	Use Case	Description
UC16	Vehicle Browse	Policies & FAQ	User reads rental regulations, cancellation policy, insurance terms, and FAQs.
UC17	Rental	Book Vehicle	Renter selects vehicle, sets rental dates/hours, reviews cost breakdown, and submits booking request to the Owner.
UC18	Rental	Payment & Deposit	Renter pays rental fee and security deposit via VNPAY/VietQR; system records the transaction and updates booking status.
UC19	Rental	Track Booking	Renter monitors booking status (Pending, Approved, Ongoing, Completed, Cancelled) from My Trips.
UC20	Rental	Cancel & Refund	Renter cancels before rental starts within the allowed window; refund is processed to wallet per cancellation policy.
UC21	Rental	Vehicle Handover	Renter and Owner complete electronic handover form with photos at pickup and return.
UC22	Rental	Rate & Review	After trip, Renter submits star rating and text review for vehicle and Owner; stored and displayed publicly.
UC23	Rental	Report Incident	Renter files urgent incident report (accident, breakdown, theft) with photo evidence; assigned to CS.
UC24	Rental	Submit Complaint	Renter or Owner submits formal dispute; CS receives evidence and mediates resolution.
UC25	Wallet	Wallet & Transactions	User views wallet balance, tops up, withdraws to bank, and reviews full transaction history.
UC26	Fleet Mgmt	Register Vehicle	Owner submits vehicle details: brand, model, plate, category, pricing, fuel type, features, images, and legal documents.
UC27	Fleet Mgmt	Manage Listings	Owner views, edits, temporarily deactivates, or removes registered vehicle listings.

(Continued from previous page)

ID	Feature	Use Case	Description
UC28	Fleet Mgmt	Confirm Re-	Owner approves or rejects incoming booking requests within the allowed response window.
UC29	Revenue	Revenue Statistics	Owner views earnings per period, per vehicle, and full transaction breakdown from completed rentals.
UC30	Admin	Manage Users	Admin views user list, searches by name/role, suspends or permanently removes policy-violating accounts.
UC31	Admin	Assign Roles	Admin assigns or modifies user roles (Renter, Owner, CS, Admin) and adjusts permissions.
UC32	Admin	Approve Vehicles	Admin reviews new listing submissions, approves or rejects them, manages categories, removes violations.
UC33	Admin	Manage Transactions	Admin monitors all payment flows, processes deposit refunds, and resolves payment disputes.
UC34	Admin	System Config	Admin manages global settings (platform fee %, cancellation timeframes, notification templates) via key-value params.
UC35	CS	Verify KYC	CS reviews identity documents pre-filled by AI OCR and approves or rejects with feedback.
UC36	CS	Approve Vehicles	CS reviews new vehicle listings from Owners and approves or rejects before going live.
UC37	CS	Support Tickets	CS receives, assigns, and responds to user support tickets about platform issues.
UC38	CS	Handle Complaints	CS receives dispute complaints, reviews evidence, mediates resolution, and closes with outcome.
UC39	CS	Handle Incidents	CS monitors emergency incident reports submitted during trips, coordinates resolution, updates status.

(Continued from previous page)

ID	Feature	Use Case	Description
UC40	CS	Moderate Re-views	CS inspects user reviews, hides/removes inappropriate content, and reports violations to Admin.
UC41	CS	Statistics Re-ports	CS compiles and submits weekly/monthly reports to Admin covering incidents, complaints, KYC stats, and violations.
UC42	AI	KYC OCR	System auto-extracts identity info (name, ID number, DOB, address) from uploaded KYC documents to pre-fill CS review.
UC43	AI	Support Chat-bot	AI chatbot on Contact page auto-answers common user queries, reducing CS first-response load.

3.1 Payment Integration

The baseline transaction calculation for a booking session is:

$$T_{\text{total}} = (P_{\text{day}} \times N_{\text{days}}) + D_{\text{deposit}} + F_{\text{service}} \quad (1)$$

where T_{total} is the total transaction cost, P_{day} is the daily rental price, N_{days} is the rental duration in days, D_{deposit} is the electronic security deposit, and F_{service} is the platform service fee.

Theorem 1. Automated Contract State Theorem. *A booking state transitions to Active if and only if (i) the Renter's KYC status is Approved and (ii) the VNPAY processor confirms successful deposit validation.*

Proof. Access control logs are audited on every state-change request. The system rejects any state transition from *Pending* to *Active* that does not satisfy both conditions simultaneously, preventing unverified entities from accessing owner financial data or vehicle assets.

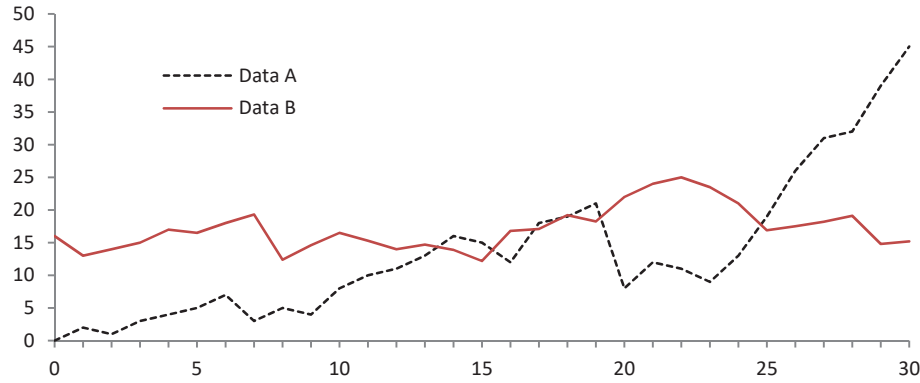
4 System High-Level Design

4.1 MVC Architecture

The platform is structured on a three-tier MVC pattern:

Table 2. Platform Functional Block Segmentation Matrix.

Block	Core Features	Primary Actors
Authentication	Account management, OAuth, KYC	All roles
Vehicle Browsing Rental & Trip Ops	Catalogue, search/filter, policies Booking, payment, handover, re-view	Guest, Renter Renter, Owner
Vehicle Management	Register, edit, confirm requests	Owner, Admin, CS
Administration	Users, roles, transactions, config	Admin
Customer Support	Tickets, complaints, incidents	CS
AI Integration	KYC OCR, support chatbot	System

**Fig. 1.** System Use Case flow rendering data transactions for target stakeholders.

- **Model** — Java POJOs and DAO classes that encapsulate SQL Server data access. Each domain entity (User, Vehicle, Booking, etc.) has a dedicated DAO implementing CRUD operations via JDBC.
- **View** — JSP pages with JSTL tag libraries responsible for rendering HTML responses. No business logic resides in the View layer.
- **Controller** — Java Servlet classes annotated with `@WebServlet` that receive HTTP requests, delegate to the appropriate service or DAO, and forward the result to the correct JSP.

All external API calls (VNPAY payment, Google OAuth 2.0, AI OCR) are encapsulated in dedicated service classes, keeping the Controller layer thin and testable.

4.2 Code Package Structure

Table 4 describes the main Java packages.

Table 4. Code Package Descriptions.

No	Package	Description
01	<code>model</code>	POJO/entity classes (User, Vehicle, Booking, Payment, etc.) mapping to database tables.
02	<code>dao</code>	Data Access Object classes implementing CRUD operations for each entity via JDBC.
03	<code>controller</code>	Servlet classes handling HTTP request routing per actor workflow (auth, rental, fleet, admin, support).
04	<code>service</code>	Business logic layer: booking validation, billing computation, refund rules, notification dispatch.
05	<code>service.payment</code>	VNPAY Sandbox API integration: payment request generation, callback verification, refund processing.
06	<code>service.ai</code>	AI KYC OCR connector and chatbot integration wrapper.
07	<code>service.auth</code>	Google OAuth 2.0 token handling, session management, OTP generation and verification.
08	<code>filter</code>	Java Servlet Filters for role-based access control, session validation, and request logging.
09	<code>util</code>	Shared utilities: email sender, file upload handler, date/time helper, JSON parser.
10	<code>config</code>	Application-level constants and <code>SystemConfig</code> key-value loader from database.

4.3 Database Design

The SQL Server relational schema consists of 28 tables. The core booking workflow links `User` $\rightarrow$ `Booking` $\rightarrow$ `Payment` $\rightarrow$ `WalletTransaction` and enforces referential integrity at the database level via foreign key constraints. Table 5 provides full descriptions of all tables.

Table 5: Database Table Descriptions.

No	Table	Description
01	<code>Role</code>	Role definitions (Guest, Renter, CarOwner, Admin, CS). PK: <code>role_id</code> .
02	<code>User</code>	Account info, personal profile, login credentials. PK: <code>user_id</code> .

(Continued from previous page)

No	Table	Description
03	UserRole	Junction table for RBAC linking users to roles. PK: user_role_id . FK: user_id , role_id , assigned_by .
04	OTPVerification	OTP codes for email verification and password recovery. PK: otp_id . FK: user_id .
05	KYC	Identity verification documents and review status. PK: kyc_id . FK: user_id , reviewer_id .
06	Brand	Vehicle brand catalogue (Toyota, Honda, VinFast, etc.). PK: brand_id .
07	VehicleCategory	Vehicle classifications (Sedan, SUV, Hatchback, MPV, etc.). PK: category_id .
08	Vehicle	Full vehicle information: specs, pricing, availability status. PK: vehicle_id . FK: owner_id , brand_id , category_id , approved_by .
09	VehicleImage	Vehicle photo gallery. PK: image_id . FK: vehicle_id .
10	VehicleDocument	Legal documents per vehicle (Registration, Insurance, Inspection). PK: doc_id . FK: vehicle_id .
11	Feature	Amenity catalogue (GPS, Dashcam, Bluetooth, child seat, etc.). PK: feature_id .
12	VehicleFeature	Junction table linking vehicles to their equipped features. PK: (vehicle_id , feature_id).
13	Booking	Rental transaction record: dates, total cost, trip status, cancellation info. PK: booking_id . FK: renter_id , vehicle_id , cancelled_by .
14	BookingHandover	Electronic handover record: vehicle condition, odometer, fuel level at pickup and return. PK: handover_id . FK: booking_id .
15	HandoverImage	Photos captured during vehicle handover events. PK: image_id . FK: handover_id .
16	Payment	Individual payment transactions (fees, deposits, refunds) via gateway or wallet. PK: payment_id . FK: booking_id , payer_id .
17	Wallet	User electronic wallet: current balance and account linkage. PK: wallet_id . FK: user_id .
18	WalletTransaction	Full ledger of wallet movements (top-up, withdrawal, payment, refund). PK: txn_id . FK: wallet_id , booking_id .
19	Review	Post-trip star ratings and text reviews for vehicle and Owner. PK: review_id . FK: booking_id , reviewer_id , vehicle_id .

(Continued from previous page)

No	Table	Description
20	Incident	Trip incidents (accident, breakdown, theft) for CS resolution. PK: <code>incident_id</code> . FK: <code>booking_id</code> , <code>reporter_id</code> , <code>assigned_to</code> .
21	IncidentImage	Photo evidence for incident reports. PK: <code>image_id</code> . FK: <code>incident_id</code> .
22	Complaint	Formal disputes between Renter and Owner for CS mediation. PK: <code>complaint_id</code> . FK: <code>booking_id</code> , <code>complainant_id</code> , <code>defendant_id</code> , <code>assigned_to</code> .
23	ComplaintEvidence	Evidence files (images, documents) attached to complaints. PK: <code>evidence_id</code> . FK: <code>complaint_id</code> .
24	SupportTicket	User support requests managed by CS. PK: <code>ticket_id</code> . FK: <code>user_id</code> , <code>assigned_to</code> .
25	TicketMessage	Message thread within a support ticket. PK: <code>message_id</code> . FK: <code>ticket_id</code> , <code>sender_id</code> .
26	Notification	System notifications dispatched to users (reminders, alerts). PK: <code>notification_id</code> . FK: <code>user_id</code> .
27	Policy	Platform policy documents (rental terms, insurance, cancellation rules). PK: <code>policy_id</code> . FK: <code>created_by</code> .
28	SystemConfig	Global key-value configuration parameters (fee %, timeframes, templates). PK: <code>config_id</code> . FK: <code>updated_by</code> .

5 Screen Flow and Authorisation

5.1 Screen Descriptions

Table 6 summarises the principal application screens grouped by feature area.

Table 6: Screen Descriptions by Feature.

#	Feature	Screen	Description
1	Auth	Register	Form collecting email, full name, password; submits to UC01 registration flow.
2	Auth	Login	Email/password form with Google OAuth button; triggers UC03/UC04.

(Continued from previous page)

#	Feature	Screen	Description
3	Auth	Reset Password	Token-validated form for setting a new password (UC06–UC07).
4	Account	Profile	View/edit personal info, avatar upload, KYC status badge (UC09).
5	Account	KYC Upload	Document upload form for National ID and Driver's Licence with AI pre-fill (UC10, UC42).
6	Account	Wallet	Balance display, top-up, withdrawal, and full transaction history (UC25).
7	Vehicle Browse	Home / Listing	Paginated vehicle cards with thumbnail, price, location; links to vehicle detail (UC13).
8	Vehicle Browse	Search & Filter	Sidebar/modal with filter controls (brand, price, category, seats, fuel type); real-time results (UC14).
9	Vehicle Browse	Vehicle Detail	Full-page vehicle view: gallery, specs, owner profile, availability calendar, reviews (UC15).
10	Vehicle Browse	Policies & FAQ	Static information page for rental terms, insurance, and FAQs (UC16).
11	Rental	Booking Form	Date/time pickers, cost summary breakdown, booking submission (UC17).
12	Rental	Payment	VNPAY/VietQR payment widget with deposit confirmation (UC18).
13	Rental	My Trips	Booking list with status chips and action buttons (cancel, view, handover) (UC19).
14	Rental	Handover Form	Electronic vehicle condition form with photo upload for pickup/return (UC21).
15	Rental	Submit Review	Star rating + text review form shown after trip completion (UC22).
16	Rental	Report Incident	Photo + description form for trip emergencies (UC23).
17	Owner	My Vehicles	List of registered vehicles with edit, deactivate, and analytics actions (UC27).

(Continued from previous page)

#	Feature	Screen	Description
18	Owner	Register Vehicle	Multi-step form: basic info, specs, pricing, features, images, documents (UC26).
19	Owner	Booking Requests	Incoming request cards with Approve/Reject actions and countdown timer (UC28).
20	Owner	Revenue Dash-board	Charts and tables for earnings per period and per vehicle (UC29).
21	Admin	User Management	Searchable user table with role badges and Suspend/Delete actions (UC30–UC31).
22	Admin	Vehicle Approvals	Pending listing queue with document viewer and Approve/Reject actions (UC32).
23	Admin	Transaction Monitor	Full payment ledger with refund processing and dispute flags (UC33).
24	Admin	System Config	Key-value settings editor for platform parameters (UC34).
25	CS	KYC Review	Document viewer with AI OCR pre-filled fields and Approve/Reject form (UC35).
26	CS	Vehicle Review	New vehicle listing queue for CS pre-approval (UC36).
27	CS	Support Tickets	Ticket inbox with priority, status, assignment, and reply thread (UC37).
28	CS	Complaints	Dispute case view with evidence gallery and mediation outcome form (UC38).
29	CS	Incidents	Active incident dashboard with trip context and status update (UC39).
30	CS	Review Moderation	Review queue with hide/remove controls and violation report (UC40).
31	CS	Reports	Summary dashboard for weekly/monthly KPI compilation (UC41).
32	AI	Support Chatbot	Floating chatbot widget on Contact page for automated FAQ responses (UC43).

5.2 Screen Authorisation Matrix

Table 7 defines role-based access to each functional area. **G** = Guest, **R** = Renter, **O** = Car Owner, **A** = Admin, **CS** = Customer Service.

Table 7. Screen Authorisation Matrix.

Screen / Action	G R O A CS
Home / Vehicle Listing (browse)	
Vehicle Detail (view)	
Policies & FAQ	
Register / Login	
Profile (view & edit own)	
KYC Upload	
Wallet & Transactions	
Booking Form / Payment	
My Trips	
Handover Form (as Renter)	
Submit Review / Incident	
Register Vehicle	
My Vehicles (own)	
Booking Requests	
Revenue Dashboard	
Handover Form (as Owner)	
User Management	
Vehicle Approvals (Admin)	
Transaction Monitor	
System Config	
KYC Review	
Vehicle Review (CS)	
Support Tickets	
Complaints	
Incidents	
Review Moderation	
CS Reports	

5.3 Non-UI Background Functions

Table 8 describes system functions with no user-facing screen (batch jobs, scheduled tasks, and background services).

6 Conclusion

This paper presented the design and implementation of a comprehensive web-based peer-to-peer self-driving car rental support platform. The system addresses critical gaps in the Vietnamese vehicle sharing market by delivering a fully digital workflow covering vehicle registration, booking, VNPAY-integrated payment, electronic handover documentation, KYC verification, and multi-tier dispute resolution. The MVC architecture built on Java Servlet, JSP, and SQL Server provides a clean separation of concerns and facilitates independent testing of business logic, data access, and presentation layers. AI augmentation via KYC OCR (UC42) and the support chatbot (UC43) measurably reduces manual processing overhead for Customer Service agents.

Experimental testing across all five stakeholder roles confirms that the 43-use-case scope is successfully implemented with correct role-based access control and consistent transaction integrity enforced by the Automated Contract State Theorem (Theorem 1).

Future work will focus on: (i) native mobile application development (Android/iOS) to extend Renter and Owner reach; (ii) a recommendation engine leveraging rental history to surface personalised vehicle suggestions; and (iii) migration of the VNPAY integration from Sandbox to the production gateway with full PCI-DSS compliance.

The project development team acknowledges the technical infrastructure and guidance provided by FPT University, Da Nang, Vietnam.

Disclosure of Interests. The authors declare no competing financial or personal interests that could have influenced the work reported in this manuscript.

References

1. VNPAY: VNPAY Sandbox Payment Gateway Integration Documentation (2026), <https://sandbox.vnpayment.vn/apis/>, last accessed 2026/06/08
2. Schildt, H.: Java: The Complete Reference. 12th edn. McGraw Hill, New York (2023)
3. Microsoft: SQL Server Relational Database Management Guidelines for Enterprise Applications (2024)
4. Le, T., Nguyen, M.: E-Commerce Software Architecture Optimization in Urban Transport Networks. *Journal of Software Engineering* 5(2), 45–58 (2025)

Table 8. Non-UI Background System Functions.

#	Feature	Function	Description
1	Booking Lifecycle	Auto-Expire Pending Bookings	Scheduled job (every 30 min) cancels Pending bookings not confirmed by the Owner within the configured response window.
2	Payment	VNPAY IPN Call-back Handler	Servlet endpoint receiving VNPAY Instant Payment Notification; validates HMAC signature, updates Payment and Booking status atomically.
3	Refund Processing	Auto-Refund Cancellation	on Triggered on booking cancellation; calculates refund amount per cancellation policy and credits the Renter's wallet.
4	Notification	Email Dispatch Service	Async service sending transactional emails (OTP, booking confirmation, payment receipt, KYC result) via SMTP.
5	Notification	In-App Notification Push	Inserts Notification records and pushes real-time alerts to connected users via server-sent events.
6	AI	KYC OCR Processor	Background service submitting uploaded identity images to the OCR API and persisting extracted fields to the KYC record for CS pre-fill.
7	Booking Lifecycle	Trip Status Scheduler	Daily cron job advancing booking status from <i>Approved</i> to <i>Ongoing</i> and from <i>Ongoing</i> to <i>Pending Return</i> based on rental start/end timestamps.
8	Reporting	Automated CS Report Generator	Weekly scheduled task aggregating KPIs (incident count, complaint resolution rate, KYC throughput) and storing summary records for CS dashboard.